ADAPTDUPLEX: FROM STATIC TO ADAPTIVE FULL-DUPLEX SPOKEN DIALOGUE

Zhiyang Zhou, Yingxin Shang, Zhou Wang, Hongwei Cai, Weixu Wang, Shuran Zhou, Shuofeng Zhao, Wenke Fan, Qingxiang Guo, Dawei Yang, Lin Yang, Yang Song*

Zuoyebang Education Technology (Beijing) Co., Ltd.

arXiv:2609.29217v3 [cs.SD] 9 Oct 2026

ABSTRACT

Full-duplex spoken dialogue requires simultaneous listening and speaking at sub-second latency, under conversational timing and cognitive demands that change moment to moment. Yet current models mostly impose static operating points, lacking a systematic mechanism for adaptive decisions. We present AdaptDuplex, which extends Qwen3-Omni with such a mechanism, co-designed across three layers. A compact token-level protocol represents every window as a canonical sequence, trains dual-stream alignment through a bounded text lead over speech, and exposes every behavioral decision as an explicit token for training-free runtime control via logits bias. Adaptive mechanisms dynamically predict among discrete window durations and augment direct response as needed with non-blocking cognitive consolidation and multi-flight external reasoning. A progressive pipeline introduces these behaviors through a three-stage Thinker curriculum, then Talker-only and joint SFT, with GRPO as a preliminary increment. On Full-Duplex-Bench v1 and v1.5, AdaptDuplex outperforms DuplexOmni on 18 of 21 comparable turn-taking, overlap-behavior, and timing metrics and MiniCPM-o 4.5 on 19 of 22, with gains in both interaction decisions and response timing. On human-recorded HumDial-FDBench, it attains the top Final score (69.6) of the compared duplex models.

***Index Terms*—** full-duplex spoken dialogue, real-time, token-level protocol, adaptive interaction, curriculum learning

1. INTRODUCTION

Human conversation is inherently full-duplex: listeners acknowledge, interrupt, and revise while the other party is still speaking. Replicating this requires a spoken dialogue model that listens continuously while generating, coordinates overlapping speech, and responds within sub-second time scales as conversational timing and cognitive demands shift from moment to moment. Dedicated duplex models have established the feasibility of such interaction [1, 2, 3, 4, 5], and strong omni-models provide a capable pretrained starting point for multimodal understanding and generation; yet a full-duplex model that adapts as these demands change remains hard to obtain.

Existing duplex models still impose a static operating point: window duration and available actions are hard-wired at design time, and the protocol is either too heavy to grow with each added decision or too sparse to carry a richer policy. DuplexOmni [6] and MiniCPM-o 4.5 [7] both fix the window duration; the former admits only single-flight external reasoning without intermediate cognitive consolidation, and the latter provides only binary interaction control. Adaptive full-duplex interaction therefore requires a real-time stream of context-dependent decisions: temporal granularity must remain selectable and available actions composable at runtime, rather than hard-wired into the protocol and training setup. The text–speech relation is likewise fixed: frame-synchronous models such as Moshi [1] act on every 80-ms frame with a time-aligned text stream, and streaming TTS interleaves text and speech tokens at a fixed ratio [8], so neither lets the text lead over speech vary within a bound.

Introducing these decisions into a pretrained omni-model presents a further challenge. Prior work has separately explored state tokens, latent cognition, and policy optimization for interaction control [9, 10, 11], but exposing a pretrained model to protocol adherence, interaction control, and speech generation all at once entangles their learning objectives, and staging training by data scale, quality, trainable modules, or half- to full-duplex mode [3] does not order individual duplex behaviors by difficulty.

We initialize AdaptDuplex from Qwen3-Omni [12], building on its pretrained multimodal capabilities rather than learning duplex competence from scratch, and co-design three layers—a compact canonical sequence protocol, adaptive interaction mechanisms, and a progressive training pipeline—around one principle: every behavioral decision is an explicit token (Fig. 1).

Our contributions are threefold:

1. We formalize a compact canonical sequence protocol for full-duplex interaction. It reduces structural decoding overhead, trains dual-stream alignment through a bounded text lead over speech, and exposes interaction decisions to training-free runtime control through logits bias.
2. We propose adaptive interaction mechanisms along two axes: dynamically predicting among discrete window durations, and conditionally augmenting direct response with cognitive consolidation and multi-flight external reasoning.
3. We develop a progressive pipeline staging duplex behaviors through a three-stage Thinker curriculum by data difficulty, then Talker-only and joint SFT, plus preliminary GRPO.

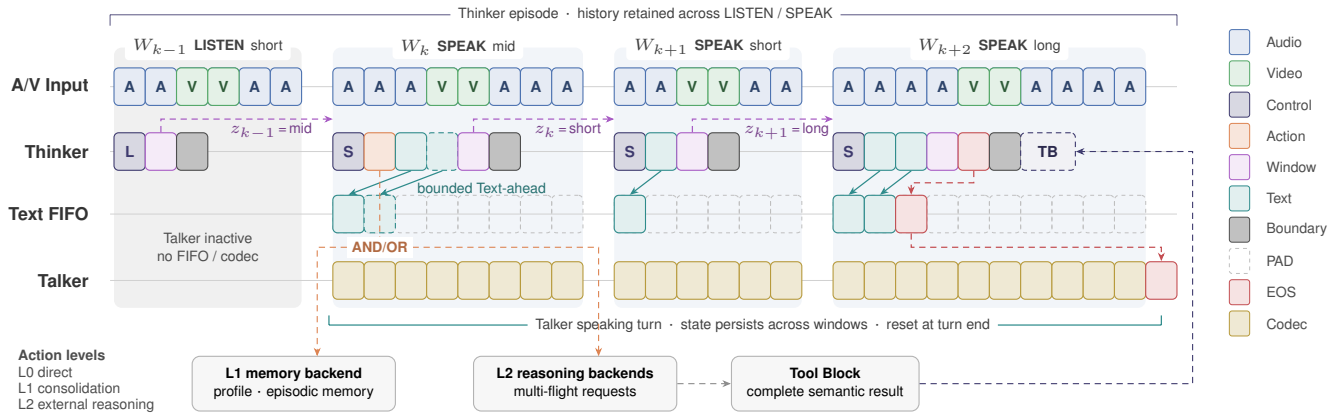
2. PROPOSED METHOD

AdaptDuplex inherits the Thinker–Talker architecture of Qwen3-Omni [12], but moves the Thinker from generating a complete response per user turn to making per-window interaction decisions throughout an episode, with reply text only as an optional output. The Talker keeps its synthesis role but changes its conditioning.

2.1. Token-level Sequence Protocol

Each window is organized as a *canonical sequence*, enclosed by Unit delimiters. The Thinker consumes the causally available environment and then emits, in a fixed order, a listen/speak control; a possibly empty sequence of action tokens drawn from <|memory|> and the indexed <|launch:N|>/<|cancel:N|> families; optional assistant text; a next-window token; and boundary tokens. The window that

*Corresponding author.



e.g., W_{k-1}*:* <|listen|> <|window:mid|> <|chunk_eos|>; W_k*:* <|speak|> <|launch:0|> Let me check. <|window:short|> <|chunk_eos|>

Fig. 1. AdaptDuplex overview: one interleaved token protocol carries adaptive windows, dual-stream alignment, and tiered reasoning. Window bands, token placements, and A/V interleaving are schematic; explicit decision tokens allow inference-time control via logits bias.

ends a speaking turn emits <|turn_eos|> before <|chunk_eos|>. The complete grammar is given in the Appendix A.1.

The canonical sequence also drives speech generation. When a <|speak|> decision starts a speaking turn, the Talker prefills once from the environment history through that Unit, instead of re-collecting user-turn segments; later Units reach the Thinker but not the current Talker. After one bootstrap, the Talker generates codec tokens autoregressively, popping one text condition per acoustic position from a turn-local FIFO that queues assistant text as the Thinker emits it, rather than consuming text as a continuous stream; <|tts_pad|> fills empty positions. Queued text carries across windows but never across speaking turns. On <|turn_eos|>, <|tts_eos|> is enqueued; after the remaining speech, the Talker predicts <|codec_eos|> and resets its turn-local state.

A. Compact serialization. DuplexOmni [6] serializes per-window metadata as JSON. We instead encode protocol fields as compact tokens in a fixed order, reducing structural decoding overhead (§3.3).

B. Dual-stream alignment. The FIFO coordinates the Thinker and Talker despite their different token rates. During training, bounded Text-ahead moves an order-preserving prefix of up to $d_k \leq d_{\max} = 2$ text tokens from each window into the preceding window of the same speaking turn. This changes only per-window allocation: the flattened speaking-turn text sequence, control decisions, codec targets, and causal boundary for future environments remain unchanged. With interleaved text/<|tts_pad|> conditioning, this yields streaming text–speech alignment with a bounded text lead without modifying the Talker architecture.

C. Runtime controllability. Because behavioral decisions are explicit tokens, additive logits bias adjusts runtime behavior at inference without modifying model weights.

2.2. Adaptive Interaction Mechanisms

The canonical sequence exposes two context-dependent runtime decisions: temporal granularity and action composition.

A. Dynamic window prediction. Fixed windows impose a global latency–evidence trade-off: at a turn boundary, a short window must decide on little post-boundary acoustic evidence, whereas a long window gathers more evidence but responds slowly. At the end of window k, the model predicts $z_k \in \{\text{short, mid, long}\}$ from only causally available multimodal context, setting window $k{+}1$ to 0.48, 0.64, or 0.96 s (6, 8, or 12 codec frames at 12.5 Hz); mid is the fallback under insufficient evidence and initializes the first window. Weak labels for z_k follow priority rules over timestamps, acoustic and semantic boundaries, and overlap labels (see Appendix B).

B. Conditional action composition. A binary choice between direct response and external reasoning overlooks an intermediate cognitive-consolidation operation. We therefore augment direct response (L0, an empty action sequence) with two independently selectable, non-blocking action families that may co-occur within a window. L1 emits <|memory|> to consolidate state through a memory backend; later Units see the update without stalling the current response. L2 manages up to four in-flight requests to a pluggable external LLM, tool, or agent via indexed <|launch:N|>/<|cancel:N|>, $N \in \{0, \ldots, 3\}$, where N names a runtime slot rather than a request category: <|launch:N|> starts an asynchronous request in idle slot N, and <|cancel:N|> cancels the request in that slot and is valid only while one exists. Valid results and explicit backend failures re-enter the Thinker at Unit boundaries as loss-masked, semantically complete *Tool Blocks*, letting the model distinguish failure from no return; cancelled, stale, or unreturned requests inject nothing, and neither a failure nor a non-return blocks the current response.

2.3. Progressive Training Pipeline

To avoid the entanglement noted in §1, we structure SFT along two orthogonal axes: data difficulty for the Thinker, and trainable modules across the Thinker–Talker stack.

A. Curriculum Thinker SFT. The Thinker is adapted in three stages of increasing duplex difficulty. Stage 1 isolates the canonical protocol on turn-based conversations under a fixed middle-window schedule, without window-token supervision; Stage 2 adds basic duplex events (backchannels, short pauses, and simple interruptions) and activates dynamic window prediction; Stage 3 introduces the full duplex data with <|memory|> and indexed <|launch:N|>/<|cancel:N|>, requiring it to compose interaction and external-reasoning orchestration actions. An automated multi-agent pipeline generates training data (Appendix D).

B. Talker and joint SFT. Talker-only SFT then freezes the Thinker and trains the Talker under FIFO text conditions with bounded Text-ahead; a final joint stage calibrates all components end to end on a smaller set of high-quality duplex conversations.

C. Policy optimization. GRPO continues from the joint-SFT check-

point as a preliminary increment, refining the interaction policy without architectural change; its reward design is in the Appendix D.6.

3. EXPERIMENTS

3.1. Experimental Setup

Model. We compare AdaptDuplex with the native full-duplex baselines DuplexOmni and MiniCPM-o 4.5 and with a half-duplex reference, Qwen3-Omni with VAD endpointing and barge-in cancellation. DuplexOmni shares our Qwen3-Omni base but draws on about 3.02M raw conversations [6], an estimated four times our speech hours; MiniCPM-o 4.5 builds on the dense Qwen3-8B, with fewer total but more active parameters than our 30B-A3B MoE (3B active). All three duplex systems stream on a single H100 GPU (baselines with official code and parameters), so latencies include computation; since Thinker, Talker, and Code2Wav run in parallel, speech starts once the first text token is decoded, without waiting for the rest of the window's output. AdaptDuplex is initialized from Qwen3-Omni-30B-A3B-Instruct and trained on 8 H100 GPUs. Bounded Text-ahead uses three lead schedules: Aligned (d_k=0, forced-alignment windows without look-ahead), Fixed Ahead (all internal leads at $d_{\max}$), and Random Ahead (a bounded random walk over d_k). Unless stated otherwise, all variants before joint SFT share one Talker from Talker-only SFT with the Random Ahead schedule; Table 3 adapts one Talker per schedule. Decoding follows the official Qwen3-Omni policy (greedy Thinker, sampled Talker).

Data. Curriculum SFT and its single-stage ablation, Mixed SFT, use an internally constructed synthetic full-duplex corpus of 765,424 dialogues (17,259 h, of which 10,857 h are assistant speech); Mixed SFT matches Curriculum in instance count and per-instance exposure on this corpus. Joint SFT continues from the Curriculum checkpoint on a smaller high-quality subset of DuplexDrama [13].

Evaluation. We use Full-Duplex-Bench (FDB) [14], its v1.5 extension (FDB-v1.5) [15] with genuine interruptions and other overlap scenarios, and the human-recorded, scripted Chinese and English HumDial-FDBench [16]. Both FDB evaluations follow their official scoring and transcription protocols. Serialization and replay share 24 streaming records; replay re-segments them under each fixed duration and the dynamic policy. Text-ahead conditions share utterances, Thinker text, and window schedules; naturalness uses the official UTMOSv2 [17]. L1/L2 action prediction is evaluated from gold prefixes, with positive-class F1 for the L1 memory action and action EM, token-level launch/cancel F1, and invalid-action rate for L2; memory instances and L2 scenarios from the same episodes use gold context. No FDB or HumDial scripts, audio, or speakers enter data generation, no setting is tuned on test sets, and benchmarking attaches no L1/L2 backend or DuplexOmni thinking layer.

3.2. Overall Full-Duplex Performance

Table 1 isolates scene judgment on the turn-taking and overlap cases: versus DuplexOmni, AdaptDuplex more often withholds a full turn when only a backchannel is due (FDB-v1 takeover rate, TOR, 0.334 vs. 0.491; MiniCPM-o 4.5 is lowest at 0.273) and resumes more reliably under backchannel and background-speech overlap (FDB-v1.5 behavior 0.55 vs. 0.28; 0.43 vs. 0.07). Its higher Candor pause TOR (0.299 vs. 0.097) is the flip side of taking the floor when due (smooth-turn TOR 0.982 vs. 0.193). These gains do not cost speed: response latency is sub-second in all reported FDB scenarios and the lowest among native duplex systems in every FDB-v1.5 scenario, and on FDB-v1 interruptions AdaptDuplex is both more accurate (TOR 0.985 vs. 0.746) and faster (0.131 vs. 0.158 s); MiniCPM-o 4.5 matches that TOR at 1.408 s.

On human-recorded interactions with the same operating point (Table 2), AdaptDuplex attains the highest Final score among the compared duplex dialogue models (69.6), with delays of 0.835 s (ZH) and 1.345 s (EN), faster than MiniCPM-o 4.5 and Duplex-

Table 1. Full-duplex interaction performance on (a) FDB-v1 turn-taking and (b) FDB-v1.5 overlap handling.

Task	Metric	Moshi†	Freeze-Omni†	Qwen3-Omni +VAD	MiniCPM-o 4.5	Duplex Omni	Adapt Duplex
(a) FDB-v1: turn-taking							
Pause	Syn. TOR↓	0.985	0.642	0.993	0.270	**0.116**	0.128
	Can. TOR↓	0.980	0.481	0.949	0.463	**0.097**	0.299
Back-channel	TOR↓	1.000	0.636	0.344	**0.273**	0.491	0.334
	Freq.↑	0.001	0.001	0.018	0.020	0.034	**0.040**
	JSD↓	0.957	0.997	0.879	0.902	0.847	**0.821**
Smooth turn	TOR↑	0.941	0.336	**0.992**	0.882	0.193	0.982
	Resp. lat.↓	0.265	0.953	3.358	1.222	0.000‡	**0.085**
Inter-ruption	TOR↑	**1.000**	0.867	0.995	0.985	0.746	0.985
	GPT-4o↑	0.765	3.615	**4.824**	4.497	4.222	4.512
	Resp. lat.↓	0.257	1.409	3.380	1.408	0.158	**0.131**
(b) FDB-v1.5: overlap handling							
Inter-ruption	Behavior↑	0.50	0.72	**0.97**	0.82	0.67	0.79
	Stop lat.↓	1.16	1.42	**0.09**	2.49	2.01	1.32
	Resp. lat.↓	1.47	1.35	3.70	1.86	0.36	**0.34**
Back-channel	Behavior↑	0.06	**0.80**	0.00	0.46	0.28	0.55
	Stop lat.↑	0.42	0.66	–	0.76	0.80	**0.84**
	Resp. lat.↓	3.00	2.16	2.73	2.34	0.82	**0.43**
Talking to others	Behavior↑	0.20	0.25	0.01	0.21	**0.37**	0.35
	Stop lat.↑	0.87	1.39	0.10	1.71	1.87	**1.89**
	Resp. lat.↓	2.38	1.32	3.16	1.85	1.95	**0.33**
Back-ground speech	Behavior↑	0.08	0.26	0.00	0.37	0.07	**0.43**
	Stop lat.↑	0.54	0.98	0.09	1.46	1.50	**1.51**
	Resp. lat.↓	1.62	1.60	3.19	1.61	0.74	**0.36**

(a): Syn./Can.: synthetic/Candor pauses; Freq. in events/s; GPT-4o scores (0–5) and latencies (s) conditional on takeover (TO=1); negative per-sample latencies (annotated turn ends lag true ones) clipped to 0. ‡All samples negative; excluded from highlighting. (b): Behavior is 1 for Respond (interruption) or Resume (other scenarios), 0.5 for Uncertain (talking-to-others, background-speech), 0 otherwise; –: no qualifying event. †Published results (FDB [14] Tab. III; FDB-v1.5 [15] Tab. 2); latencies not hardware-matched. Abstract win counts exclude ‡ and treat ties as non-wins.

Table 2. HumDial-FDBench evaluation on Chinese (ZH) and English (EN) human-recorded interactions.

Metric	Split	Moshi†	Freeze-Omni†	Qwen3-Omni +VAD	MiniCPM-o 4.5	Duplex Omni	Adapt Duplex
Int.↑ (%)	ZH	NR	NR	**91.3**	81.8	56.4	90.4
	EN	NR	NR	**80.9**	63.0	55.2	74.4
	ALL	35.4	29.6	**86.1**	72.4	55.8	82.2
Rej.↑ (%)	ZH	NR	NR	36.8	44.4	40.0	**51.8**
	EN	NR	NR	34.2	**49.6**	47.8	46.1
	ALL	22.8	**50.2**	35.6	47.2	44.1	49.1
Delay↓ (s)	ZH	NR	NR	4.670	2.074	4.266	**0.835**
	EN	NR	NR	4.924	2.315	4.590	**1.345**
	ALL	2.876	2.578	4.771	2.203	4.459	**1.042**
D-Sco.↑	ALL	56.3	59.5	41.9	64.0	43.8	**85.4**
Final↑	ALL	34.5	43.8	57.1	60.6	48.7	**69.6**

NR: not reported. ALL pools ZH and EN under the official protocol. Int./Rej.: averaged subscenario scores; Delay: combined applicable latency types. D-Sco.: official logarithmic normalization of ALL Delay; Final: 0.4 Int. + 0.4 Rej. + 0.2 D-Sco. †From HumDial-FDBench [16] Tab. 2.

Table 3. Talker text–speech alignment under the three Text-ahead lead schedules (1,516 ZH + 1,514 EN paired utterances).

Training	UTMOS v2↑	CER/WER ↓(%)	ValidEnd ↑(%)	EarlyCut ↓(%)	Late5+ ↓(%)	NoEOS ↓(%)
Aligned	2.90	10.74/19.10	50.00	0.00	8.02	0.36
Fixed Ahead	3.55	5.38/10.03	84.75	0.00	1.85	0.07
Random Ahead	3.65	4.83/8.93	86.37	0.00	1.49	0.03

Entries: ZH CER / EN WER (%, Qwen3-ASR-1.7B). EOS offsets in codec positions. ValidEnd: complete by final-window boundary; EarlyCut: truncated; Late5+: $[5, 16)$; NoEOS: $[16, +\infty)$; rest: $[1, 5)$.

Omni. The half-duplex Qwen3-Omni+VAD remains more accurate at interruption (86.1 vs. 82.2 ALL), whereas AdaptDuplex is more accurate at rejection (49.1 vs. 35.6 ALL).

3.3. Protocol Efficiency, Alignment, and Control

To test serialization, we hold the semantic records, the fixed 0.48-s window, and the adapted Talker constant and replace only the canonical sequence with a matched DuplexOmni-style JSON encoding of the same fields on 3,724 paired windows. Canonical serialization cuts tokens per window from 26.440 to 5.867 (-78%) and the protocol's share of output tokens from 93.88% to 68.70%, removes structurally invalid windows (0.48% to 0.00%), and lowers the per-window GPU busy time (union of Thinker, Talker, and Code2Wav) at P50/P90/P99 from 789.4/957.6/1,164.2 ms to 418.5/491.2/672.2 ms, i.e., by 47%/49%/42% (percentiles conditional on valid completion).

Table 3 tests dual-stream alignment, varying only the Text-ahead lead schedule. Any look-ahead helps sharply over Aligned: Fixed Ahead and Random Ahead raise pooled UTMOSv2 from 2.90 to 3.55 and 3.65, roughly halve bilingual CER/WER, and lift ValidEnd from 50.00% to 84.75% and 86.37%. Random Ahead, the default Talker schedule, is best on naturalness, error, and ValidEnd.

Table 4(b) tests runtime control by adding $\ln \alpha$ to the speak logit ($\alpha \in \{0.5, 1, 2\}$; $\times 1$ is the untuned policy) at the Curriculum checkpoint, with the Thinker, Talker, window policy, and test inputs fixed. Raising α from 0.5 to 2 moves the operating point monotonically: interruption behavior rises (0.76→0.81) and response latency shortens (0.42→0.33 s), while backchannel, talking-to-others, and background-speech scores fall and yield slows slightly (1.28→1.36 s)—an inference-time listen/speak trade-off.

3.4. Adaptive Interaction

Dynamic window. Table 4(a) runs the Default checkpoint with each window duration fixed at inference via window-token bias and makes the fixed-window dilemma explicit: the short window is fastest but lowest on backchannel, whereas the long window is best on backchannel and background speech but slowest, and its longest hold comes with the slowest yield. The dynamic policy stays within reach of each extreme at once—near-short latency (0.37 s, 1.32 s) with near-long backchannel (0.55)—and is the single best setting on talking-to-others (0.35). Controlled autoregressive replay shows the policy is not degenerate: at 43.9 GPU-s/min (busy time of one H100 per input minute, i.e., $0.73\times$ real time) it sits between always-short (48.3) and always-long (33.5), so it is not defaulting to the longest, cheapest window. The SPEAK/LISTEN breakdown, per-stage results, and window-policy matrices are in the Appendix E.

Tiered actions. From gold prefixes, the model predicts the sparse L1 <|memory|> action at F1 80.66% and L2 orchestration at action EM 76.84% (exact match of the executed action set) and launch F1 68.63%, with cancellation the weakest sub-decision (F1 54.06%); concurrency is the main stressor, with EM falling to 65.17% and the invalid-action rate nearly doubling (3.80%→7.10%) when at least two requests are in flight. With gold PROFILE and MEMORY context, L1 fact accuracy rises from 10.25% to 62.75%, so the Thinker uses injected consolidated state. For L2, canary accuracy with gold results (all target and no forbidden canaries in the Thinker text) is 86.50/73.50/68.50% for single, distracted, and composed queries, and 81.50/80.50% under injected failure and silent non-return.

Table 4. Controlled variations of the default system (Curriculum SFT checkpoint, dynamic window, no logit bias) on FDB-v1.5. (a) and (b) change one inference-time setting of the same checkpoint; (c) changes the training recipe.

Variant	Behavior				Timing (s)		
	Int.↑	Back.↑	Talk.↑	Bg.↑	Yield↓	Hold↑	Resp.↓
Default	0.79	0.55	0.35	0.43	1.32	1.41	0.37
(a) Window policy							
Fixed 0.48 s	0.82	0.52	0.31	0.34	1.19	1.38	0.32
Fixed 0.64 s	0.78	0.54	0.33	0.43	1.81	1.40	0.45
Fixed 0.96 s	0.76	0.56	0.34	0.46	2.61	1.46	0.66
(b) Speak-logit scale α							
$\times 0.5$	0.76	0.58	0.38	0.46	1.28	1.34	0.42
$\times 2.0$	0.81	0.49	0.32	0.37	1.36	1.48	0.33
(c) Training recipe							
Mixed SFT	0.62	0.41	0.28	0.24	2.86	1.40	1.65
Default + Joint	0.83	0.55	0.37	0.46	1.20	1.44	0.32
+ GRPO	0.85	0.57	0.44	0.45	1.08	1.45	0.30

Behavior uses the scenario mapping of Table 1(b). Yield = interruption stop latency; Hold = macro-averaged stop latency over the three non-interruption scenarios; Resp. = macro-averaged response latency over all four scenarios; both macro-averages use equal scenario weights. All rows: free-running end-to-end evaluation. (b): additive $\ln \alpha$ bias on the speak logit ($\times 1 =$ Default, no bias). (c): Mixed SFT is the matched-exposure single-stage control; + GRPO continues from Default + Joint.

3.5. Training Strategy

Table 4(c) compares training recipes, all with the dynamic window policy; Mixed SFT is the matched-exposure single-stage control. Curriculum accounts for most of the gain over Mixed SFT, most visibly on latency (response 1.65→0.37 s), and improves behavior in all four scenarios. Joint SFT (from Curriculum) and GRPO (from Joint) add mostly monotonic changes, clearest for talking-to-others (0.35→0.44), yield, and response; backchannel is unchanged after Joint SFT, and background-speech dips slightly after GRPO. Other results, including Tables 1–2, use the Curriculum checkpoint, which Joint SFT matches or improves on every Table 4 metric; further protocol-cost and L1/L2 analyses are in the Appendix E.

4. CONCLUSION

We presented AdaptDuplex, which extends Qwen3-Omni by co-designing a compact token-level protocol, adaptive window and action mechanisms, and a progressive training pipeline.

Several limitations remain: evaluation currently covers Chinese and English only, the window controller selects among three discrete durations rather than a continuous horizon, and external reasoning is bounded to four concurrent requests. General-ability retention after duplex adaptation and end-to-end L1/L2 evaluation with real backends remain to be measured. Future work will study continuous window prediction, more languages, adaptive concurrency limits, training and evaluation with visual inputs, and multi-party conversation.

5. ACKNOWLEDGMENT

The authors used Claude (Anthropic, Claude Sonnet 5) to improve the language and grammar of the manuscript and to obtain suggestions on the organization of Sections 1 and 2. All technical content, experiments, and conclusions were produced by the authors, who reviewed and take full responsibility for the final text.

6. REFERENCES

[1] Alexandre Défossez, Laurent Mazaré, Manu Orsini, Amélie Royer, Patrick Pérez, Hervé Jégou, Edouard Grave, and Neil Zeghidour, "Moshi: a speech-text foundation model for real-time dialogue," 2024.

[2] Ziyang Ma, Yakun Song, Chenpeng Du, Jian Cong, Zhuo Chen, Yuping Wang, Yuxuan Wang, and Xie Chen, "Language model can listen while speaking," 2024.

[3] Qinglin Zhang, Luyao Cheng, Chong Deng, Qian Chen, Wen Wang, Siqi Zheng, Jiaqing Liu, Hai Yu, Chaohong Tan, Zhihao Du, and Shiliang Zhang, "OmniFlatten: An end-to-end GPT model for seamless voice conversation," 2025.

[4] Wenyi Yu, Siyin Wang, Xiaoyu Yang, Xianzhao Chen, Xiaohai Tian, Jun Zhang, Guangzhi Sun, Lu Lu, Yuxuan Wang, and Chao Zhang, "SALMONN-omni: A codec-free LLM for full-duplex speech understanding and generation," 2024.

[5] Qingkai Fang, Shoutao Guo, and Yang Feng, "BayLing-Duplex: Native full-duplex speech dialogue with a single autoregressive LLM," 2026.

[6] Muye Huang, Lingling Zhang, Xingyu Yu, Lei Shi, Zhanyu Ma, Jun Xu, Jiuchong Gao, Jinghua Hao, Renqing He, and Jun Liu, "DuplexOmni: Real-time listening, seeing, thinking, and speaking for full-duplex interaction," 2026.

[7] Junbo Cui, Bokai Xu, Chongyi Wang, Tianyu Yu, Weiyue Sun, Yingjing Xu, Tianran Wang, Zhihui He, Wenshuo Ma, Tianchi Cai, Jiancheng Gui, Luoyuan Zhang, Xian Sun, Fuwei Huang, Moye Chen, Zhuo Lin, Hanyu Liu, Qingxin Gui, Qingzhe Han, Yuyang Wen, Huiping Liu, Rongkang Wang, Yaqi Zhang, Hongliang Wei, Chi Chen, You Li, Kechen Fang, Jie Zhou, Yuxuan Li, Guoyang Zeng, Chaojun Xiao, Yankai Lin, Xu Han, Maosong Sun, Zhiyuan Liu, and Yuan Yao, "MiniCPM-o 4.5: Towards real-time full-duplex omni-modal interaction," 2026.

[8] Zhihao Du, Yuxuan Wang, Qian Chen, Xian Shi, Xiang Lv, Tianyu Zhao, Zhifu Gao, Yexin Yang, Changfeng Gao, Hui Wang, Fan Yu, Huadai Liu, Zhengyan Sheng, Yue Gu, Chong Deng, Wen Wang, Shiliang Zhang, Zhijie Yan, and Jingren Zhou, "CosyVoice 2: Scalable streaming speech synthesis with large language models," 2024.

[9] Xiong Wang, Yangze Li, Chaoyou Fu, Yunhang Shen, Lei Xie, Ke Li, Xing Sun, and Long Ma, "Freeze-Omni: A smart and low latency speech-to-speech dialogue model with frozen LLM," 2024.

[10] Donghang Wu, Tianyu Zhang, Yuxin Li, Hexin Liu, Chen Chen, Eng Siong Chng, and Yoshua Bengio, "The silent thought: Modeling internal cognition in full-duplex spoken dialogue models via latent reasoning," 2026.

[11] Yuxin Li, Donghang Wu, Guan-Ting Lin, Hung-yi Lee, Chengwei Qin, Zhehuai Chen, and Chen Chen, "Decoupling conversational dynamics in full-duplex spoken models through reinforcement learning," 2026.

[12] Jin Xu, Zhifang Guo, Hangrui Hu, Yunfei Chu, Xiong Wang, Jinzheng He, Yuxuan Wang, Xian Shi, Ting He, Xinfa Zhu, Yuanjun Lv, Yongqi Wang, Dake Guo, He Wang, Linhan Ma, Pei Zhang, Xinyu Zhang, Hongkun Hao, Zishan Guo, Baosong Yang, Bin Zhang, Ziyang Ma, Xipin Wei, Shuai Bai, Keqin Chen, Xuejing Liu, Peng Wang, Mingkun Yang, Dayiheng Liu, Xingzhang Ren, Bo Zheng, Rui Men, Fan Zhou, Bowen Yu, Jianxin Yang, Le Yu, Jingren Zhou, and Junyang Lin, "Qwen3-Omni technical report," 2025.

[13] Qingxiang Guo, Wenke Fan, Shuofeng Zhao, Dawei Yang, Zhiyang Zhou, Yingxin Shang, Hongwei Cai, Zhou Wang, Weixu Wang, Lin Yang, Shuran Zhou, and Yang Song, "DuplexDrama: A synthesized dialogue dataset with scenarios, full-duplex behaviors, expressive speech, and sound events," 2026.

[14] Guan-Ting Lin, Jiachen Lian, Tingle Li, Qirui Wang, Gopala Anumanchipalli, Alexander H. Liu, and Hung-yi Lee, "Full-Duplex-Bench: A benchmark to evaluate full-duplex spoken dialogue models on turn-taking capabilities," 2025.

[15] Guan-Ting Lin, Shih-Yun Shan Kuan, Qirui Wang, Jiachen Lian, Tingle Li, Shinji Watanabe, and Hung-yi Lee, "Full-Duplex-Bench v1.5: Evaluating overlap handling for full-duplex speech models," 2026.

[16] Chengyou Wang, Hongfei Xue, Guojian Li, Zhixian Zhao, Shuiyuan Wang, Shuai Wang, Xin Xu, Hui Bu, and Lei Xie, "Full-duplex interaction in spoken dialogue systems: A comprehensive study from the ICASSP 2026 HumDial challenge," 2026.

[17] Kaito Baba, Wataru Nakata, Yuki Saito, and Hiroshi Saruwatari, "The T05 system for the VoiceMOS challenge 2024: Transfer learning from deep image classifier to naturalness MOS prediction of high-quality synthetic speech," 2024.

A. PROTOCOL SPECIFICATION

A.1. Sequence grammar

```
Episode ::= Prefix (ToolBlock* Unit)+
Unit ::= UNIT_START Environment ASRView
Decision UNIT_END
ASRView ::= ASR ASRPayload ASR_EOS | ϵ
Decision ::= Listen | ContinueSpeak |
FinalSpeak
Listen ::= LISTEN NextWindow CHUNK_EOS
ContinueSpeak ::= SPEAK Action Text
NextWindow CHUNK_EOS
FinalSpeak ::= SPEAK Action Text NextWindow
TURN_EOS CHUNK_EOS
Action ::= MEMORY | LAUNCH:N | CANCEL:N
           | CANCEL:N LAUNCH:N | ϵ,
N ∈ {0, 1, 2, 3}
NextWindow ::= WINDOW_SHORT | WINDOW_MID |
WINDOW_LONG
Text ::= TextToken*
ToolBlock ::= IM_START tool ToolPayload IM_END
```

`NextWindow` schedules the following Unit rather than changing the current one, so both LISTEN and SPEAK end with a window token and <|chunk_eos|>. A speaking turn is a maximal sequence of SPEAK Units terminated by `FinalSpeak`; adjacent speaking turns need not be separated by LISTEN. SPEAK and text presence are orthogonal, so `Text` may be empty. Tool Blocks occur only at Unit boundaries, and trailing Tool Blocks without a following Unit are invalid.

Because every behavioral decision is an explicit token, additive logits bias changes runtime behavior without modifying weights. Table 5 lists the three uses.

Table 5. Logits-bias controls. Each adds a constant to the named token's logit at inference.

Bias target	Effect
<|listen|> vs. <|speak|>	Shift toward listening or speaking
An action token, set to $-\infty$	Disable that action
<|window:short|>, <|window:mid|>, <|window:long|>	Select the next-window duration

A.2. Token roles and supervision

A.3. State lifetimes

Thinker history persists across LISTEN/SPEAK transitions and speaking turns. Talker state persists across Units only within one speaking turn and is reset at its terminal SPEAK; a turn reset never clears Thinker episode history.

A.4. Turn bootstrap and text FIFO

Each speaking turn owns one Talker row and one FIFO. Before its first acoustic position, the Talker inserts the five target-free bootstrap conditions in Table 8; continuation Units do not repeat them.

When a SPEAK Unit arrives, its assistant-text tokens are appended in order. A final Unit also appends <|tts_eos|>. Every attended acoustic position pops one text condition from the left; an empty queue supplies <|tts_pad|>. Unconsumed text therefore carries across Units within the same speaking turn, but neither the FIFO nor Talker KV state crosses a turn boundary. ASR, control, action, window, boundary, and Tool-Block tokens are never enqueued. If the acoustic slots of a turn cannot consume the remaining payload and the final <|tts_eos|>, the turn is rejected rather than built with truncated text.

After the bootstrap, the first Unit combines the first FIFO condition with codec BOS and predicts its first real frame. A continuation Unit has no synthetic restart: its predecessor's last real codec frame stays pending, and after the new Unit's text is enqueued that frame combines with the next FIFO condition to predict the new Unit's first frame. The pending frame remains a supervised real frame. No codec BOS, codec PAD, or synthetic boundary is inserted between ordinary Units.

Ending the turn takes two further steps, and neither produces speech. Thinker <|turn_eos|> only marks the decision; it enqueues <|tts_eos|> so the Talker learns that no text remains. Once the remaining speech has been consumed, the last real codec frame predicts <|codec_eos|>; this is the same stopping condition in training and runtime, with no separate <|codec_pad|>-conditioned EOS position. The EOS prediction occupies no acoustic frame and carries no residual-code target, so it supervises where speech stops rather than what the next frame sounds like. At runtime <|codec_eos|> is masked in every non-final Unit, so speech cannot stop mid-turn.

Table 6. Canonical Thinker ranges and training roles. "Masked" means that the range remains in causal context but has no Thinker target.

Range	Role	Target
Prefix, Tool Block	Episode/external context	Masked
Environment, Unit delimiters	Causal input/boundary	Masked
ASR Block	Optional ASR training view	If enabled
LISTEN/SPEAK	Interaction control	Valid
Action tokens	MEMORY and indexed L2 actions	Valid
Assistant text	Reply content	Valid
Window token	Schedule following Unit	Valid
TURN/CHUNK EOS	Turn/Unit speech boundary	Valid

Table 7. State ownership across the three protocol lifetimes.

Lifetime	Retained state	Reset boundary
Episode	Thinker prefix, causal history, committed Units	Episode close
Unit	Current environment and canonical decision	Commit to history
Speaking turn	Talker KV, text FIFO, codec/PCM rolling state	Final SPEAK

Table 8. Talker bootstrap at the start of each speaking turn.

Position	Text channel	Codec channel
1	<|tts_pad|>	codec-nothink
2	<|tts_pad|>	codec-think-BOS
3	<|tts_pad|>	codec-think-EOS
4	<|tts_pad|>	speaker identity
5	<|tts_bos|>	codec-pad

A final Unit usually holds fewer real frames than its window budget, since a speaking turn rarely ends on a window boundary, so it may predict <|codec_eos|> before that budget runs out. Only if the budget runs out first does one extra non-acoustic step check the EOS prediction under the same last-real-frame condition; anything other than <|codec_eos|> is logged and dropped. The chain is therefore <|turn_eos|>→<|tts_eos|>→<|codec_eos|>.

A.5. Text-ahead transformation

For each speaking turn, let T_k be the ordered assistant-text tokens assigned to its k-th Unit. At an internal boundary k, Text-ahead samples a lead $d_k \in [0, d_{\max}]$, with $d_{\max} = 2$ in the reported experiments, and moves the first $\min(d_k, |T_{k+1}|)$ tokens of T_{k+1} to the tail of T_k. Transfers use the original per-Unit lists and occur only between adjacent Units, so applying all boundaries preserves the flattened turn sequence exactly:

$$T_1 \| \cdots \| T_K = T'_1 \| \cdots \| T'_K. \tag{1}$$

The transformation changes only when a token enters the Talker FIFO. It does not change token order or count, LISTEN/SPEAK and action decisions, codec targets, or the causal boundary of future environments. Tokens never cross a speaking-turn boundary. A turn with inconsistent Unit-level supervision masks is left unchanged, because moving a token would change its training ownership.

The random-ahead schedule draws (d_k) from a bounded random walk whose transitions increase, hold, decrease, or reset the current lead, with weights $(3, 2, 2, 1)$; the fixed-ahead schedule sets all internal leads to $d_{\max}$. The transformation is applied to an eligible turn with probability $p_{\mathrm{TA}} = 0.8$ and is disabled when $d_{\max} = 0$.

B. ADAPTIVE WINDOW POLICY

B.1. Prediction target and configurations

At the end of window k, the Thinker predicts a three-way token z_k that schedules window $k{+}1$, using only multimodal context causally available through the boundary of window k. SHORT, MID, and LONG are an ordered temporal policy, not durations intrinsic to the method. The experimental configuration instantiates them as in Table 9; every duration holds an integer number of 12.5-Hz codec frames. MID initializes the first window and is the fallback when no earlier rule applies.

Table 9. Window configurations used in the experiments.

Token	Duration (s)	Codec frames
SHORT	0.48	6
MID	0.64	8
LONG	0.96	12

B.2. Weak-label priority rules

Weak targets are assigned by a deterministic script over dataset metadata: timestamped acoustic voice-activity boundaries, a complete/incomplete semantic label on each speaking-to-silence boundary, an interruption/backchannel label on each overlap, and the ASR transcript with word timestamps. These annotations are used only to build the target. Training exposes the resulting window token, and inference predicts it from the Thinker state without the annotations.

The script returns at the first matching rule, so priority is the rule order and no separate tie-breaking step is needed. The speech-rate test compares the transcript-derived rate r against a language-specific threshold: τ_{zh}=3.5 Chinese characters/s and τ_{en}=2.0 English words/s, each about $0.7\times$ its conversational rate, so ordinary speech selects MID and only distinctly slow speech selects LONG.

Algorithm 1 Next-window weak target at a window boundary.

IF the window carries an interruption annotation **or** the transcript contains a correction marker (不对, 等等, *wait*, *actually*) THEN
  RETURN SHORT
IF a speaking-to-silence boundary falls in the window THEN
  RETURN SHORT IF its semantic label is complete, ELSE MID
IF the transcript contains a hesitation marker (嗯, 那个, *um*) THEN
  RETURN SHORT
IF silence is sustained THEN
  RETURN MID IF accumulated silence < 1.0 s, ELSE LONG
IF a silence-to-speaking transition falls in the window THEN
  RETURN MID
IF speech is sustained THEN
  RETURN MID IF $r > \tau_{\mathrm{lang}}$, ELSE LONG
IF the window carries a backchannel annotation THEN
  RETURN the preceding window's target
RETURN MID

C. TIERED ACTIONS AND ASYNCHRONOUS REQUESTS

C.1. Action levels

L0, L1, and L2 are action levels, not token names. L0 is the direct response and emits no action token. L1 emits <|memory|>. L2 uses indexed <|launch:N|> and <|cancel:N|>, $N \in \{0, 1, 2, 3\}$, to control up to four external requests. Action selection is independent of the LISTEN/SPEAK control: action tokens, when present, follow <|speak|> and precede assistant text, as in the grammar of Appendix A.1. Table 10 summarizes the three paths. None of them blocks the current response.

Table 10. Action levels and their execution paths.

Level	Action token	Execution	Blocking
L0	None	Thinker response	No
L1	<|memory|>	Local backend	No
L2	<|launch:N|>, <|cancel:N|>	External backend	No

C.2. L1 local state

The minimal L1 backend keeps stable user attributes and preferences in PROFILE and episodic conversational facts in MEMORY. The parameter-free <|memory|> trigger asks the backend to update that state from recent dialogue; it does not carry field values inside the canonical sequence. The current reply continues without waiting. A successful update is visible to later Units, and a failed update neither blocks the current response nor changes the previously committed state. A richer backend may replace this one provided it keeps the same trigger and timing.

C.3. L2 slot lifecycle

Each of the four slots is independently idle or bound to one in-flight request. Table 11 gives the legal per-slot transitions. NONE is the absence of an L2 action and leaves every slot unchanged. An invalid action is rejected rather than reinterpreted: launch requires an idle slot and cancel requires an occupied one.

Table 11. Per-slot L2 transitions at a Unit decision.

Slot state	Action	Effect	Next state
Idle	NONE	No external action	Idle
Idle	LAUNCH:N	Launch new g	Active(g)
Active(g)	NONE	Keep request	Active(g)
Active(g)	CANCEL:N	Cancel g	Idle
Active(g)	CANCEL:N; LAUNCH:N	Replace g with g'	Active(g')

Requests run while full-duplex interaction continues. A cancel invalidates the slot's active identity before cancellation finishes, so a result that races the cancellation cannot re-enter as current. Same-slot replacement cancels the old identity before launching the new one. A result is accepted only when its slot, request identity, and stage index are still current; duplicates and out-of-order results are discarded.

C.4. Return path

A current request may return one result or an ordered sequence of stages. Each stage is a complete semantic unit; partial token fragments are never shown to the Thinker. An accepted stage, whether a valid result or an explicit backend failure, is serialized as a `tool`-role Tool Block, given the next stage index, and inserted at a later Unit boundary. As defined in Appendix A, the block is loss-masked but stays in the causal context, which is how the Thinker distinguishes a reported failure from silence. Cancelled, stale, and unreturned requests insert nothing. Tool Blocks enter the Thinker context only and are not queued in the Talker FIFO. No return path blocks the current response. The protocol specifies no production timeout or retry; an expired or failed backend is reported, when it is reported, through the failure block above.

D. PROGRESSIVE TRAINING DETAILS

The data-difficulty axis (C1/C2/C3) is orthogonal to the parameter axis (Thinker-only, Talker-only, joint): only Thinker SFT traverses the curriculum, while the Talker-only and joint stages each use one fixed data mix.

D.1. Data generation pipeline

Training episodes come from an automated multi-agent pipeline. Text-only dialogue outlines are generated by orchestrator and participant LLM agents, then rendered into synthetic speech with timestamp alignment. Quality control combines ASR round-trip verification, overlap sanity checks, and human spot audits at each curriculum stage. The pipeline is the subject of a companion study; this section states only what the training setup consumes.

D.2. Thinker curriculum SFT

Table 12 lists the three stages. Duplex coverage increases from C1 to C3, and the stage episode sets are disjoint, so a later stage does not reuse an earlier stage's episodes and no mixing weight is involved. Each stage runs one epoch at a global batch of 256; a trailing partial batch counts as one update.

Table 12. Thinker curriculum stages.

	C1	C2	C3
Scope	Turn-based	Basic duplex	Full duplex
Window sup.	None (fixed MID)	Dynamic	Dynamic
Actions	None	None	L1 and L2
Duplex events	None	BC, pause, simple int.	All events
Episodes	63,659	190,976	509,269
Updates	249	746	1,990

D.3. Talker training objective

Talker-only SFT freezes the Thinker and trains the Talker together with its residual CodePredictor. Teacher forcing feeds the ground-truth codec sequence, so each position conditions on the preceding real frames. The main head predicts group 0 of the current frame; the CodePredictor predicts residual groups 1–15, giving fifteen residual targets per frame. As specified in Appendix A, the final position of a speaking turn predicts <|codec_eos|> from the last real frame and carries no residual target. No curriculum is used: codec generation conditioned on text and Thinker context is the same task at every difficulty.

D.4. Loss weighting and joint SFT

A per-token-class weight scales the loss numerator; the denominator counts supervised tokens with no weighting. Both the weighted sum and the count are reduced across data-parallel workers before the division. Tool Block tokens are always masked, since they enter from an external source and serve only as context. Table 13 gives the weights.

Table 13. Per-token-class loss weights.

Token class	Thinker-only	Joint
Control (<|listen|>/<|speak|>)	2.0	2.0
Window token	1.5	1.5
Action token	1.5	1.5
Assistant text	1.0	1.0
Boundary token	4.0	4.0
ASR view (when enabled)	1.0	1.0
Tool Block	masked	masked

The weight scales the loss numerator. The denominator counts supervised tokens without weighting.

Let S be a weighted sum of per-token losses and N the matching unweighted count. Joint SFT continues on a smaller high-quality set with every component trainable:

$$\begin{aligned} \mathcal{L}_{\text{thinker}} &= S_{\text{thinker}}/N_{\text{thinker}}, \\ \mathcal{L}_{\text{talker}} &= S_{\text{main}}/N_{\text{main}} + 0.3\, S_{\text{res}}/N_{\text{res}}, \\ \mathcal{L}_{\text{joint}} &= \mathcal{L}_{\text{talker}} + 3\, \mathcal{L}_{\text{thinker}}. \end{aligned} \tag{2}$$

D.5. Optimization

Table 14 collects the optimizer setup. The three SFT stages share it; GRPO does not inherit it.

D.6. Policy optimization (GRPO)

GRPO continues from the joint-SFT checkpoint and updates the interaction policy without changing the architecture. For each episode,

Table 14. Optimizer setup for the SFT stages and GRPO.

	SFT	GRPO
Optimizer	Adam (β_1=0.9, β_2=0.999)	Adam (β_1=0.9, β_2=0.95)
Peak learning rate	1×10^{-4}	1×10^{-5}
Global batch size	256	8
Precision	bfloat16	bfloat16
Hardware	8× NVIDIA H100	8× NVIDIA H100
LR schedule	cosine	cosine
Warmup ratio	0.05	0
Max sequence length	32,768	32,768
Gradient clipping	1.0	1.0
Weight decay	0.01	0.1

G rollouts are sampled and the advantage of each is its reward relative to the others in the group, so no value function is trained.

The reward has three separately evaluated dimensions.

- **Conversational rhythm.** Penalties for late responses, premature interruptions, and missed backchannels, computed from the streaming timestamps with no human annotation.
- **Response quality.** Speech-stream naturalness from a UTMOSv2 five-fold ensemble, plus text-stream relevance from a reference-free LLM judge.
- **L2 action appropriateness.** An LLM judge scores whether each <|launch:N|> and <|cancel:N|> is justified by the preceding context. L0 and L1 are the implicit baselines, so unnecessary and missed triggers are both penalized.

The composite reward is $\alpha_1 r_{\text{rhythm}} + \alpha_2 r_{\text{quality}} + \alpha_3 r_{\text{L2}}$. Table 15 lists the GRPO-specific quantities.

Table 15. GRPO configuration.

Quantity	Value
Rollouts per episode G	8
Reward weights $\alpha_1, \alpha_2, \alpha_3$	equal
KL coefficient	0

E. EXPERIMENTAL DETAILS AND ADDITIONAL RESULTS

E.1. Complete full-duplex results

Table 18 gives the per-scenario results behind the aggregated main-text numbers, for every reported training stage and window policy. Panel (a) is FDB-v1 and panel (b) is FDB-v1.5.

E.2. Serialization and window-policy execution

Table 19 reports the token and decoding costs of the serialization formats and the per-window costs of the window policies, including the SPEAK/LISTEN decomposition.

E.3. Text-ahead EOS distributions

Table 16 breaks the EOS-offset distribution down by language for the three Text-ahead schedules. The offset bins are $[0, 1)$ for ValidEnd, $[1, 3)$ and $[3, 5)$ for the intermediate late detail, $[5, 16)$ for Late5+, and $[16, +\infty)$ for NoEOS.

E.4. L1 and L2 diagnostics

Action prediction and information utilization share the 1,000 evaluation episodes but score different views. Action prediction expands the episodes into 156,376 jointly labelled L1/L2 decision points and uses gold prefixes. Information utilization instead supplies gold memory or Tool-Block context, so it scores the use of information rather than the decision to request it. Table 17 reports the action decisions and Table 20 the utilization results.

Table 16. Text-ahead speech quality and EOS-offset distributions.

(a) Language-specific speech quality

Group	Lang.	N	UTMOSv2↑	Mixed UTMOSv2↑	CER (%)↓	WER (%)↓
Aligned	ZH	1,516	$2.792\pm.079$	$2.902\pm.145$	10.74	–
	EN	1,514	$3.012\pm.107$		–	19.10
Fixed Ahead	ZH	1,516	$3.428\pm.043$	$3.555\pm.135$	5.38	–
	EN	1,514	$3.682\pm.051$		–	10.03
Random Ahead	ZH	1,516	$3.513\pm.077$	$3.648\pm.160$	4.83	–
	EN	1,514	$3.783\pm.092$		–	8.93

(b) Termination-offset distribution

Group	Lang.	N	$[0,1)$ (%)	$[1,3)$ (%)	$[3,5)$ (%)	$[5,16)$ (%)	$[16,+\infty)$ (%)	Mean trunc. tokens
Aligned	ZH	1,516	51.25	37.86	8.71	2.18	0.00	0.9321
	EN	1,514	48.75	22.59	14.07	13.87	0.73	1.9478
Fixed Ahead	ZH	1,516	87.27	10.82	1.52	0.40	0.00	0.2078
	EN	1,514	82.23	9.58	4.76	3.30	0.13	0.5436
Random Ahead	ZH	1,516	90.04	8.71	1.12	0.13	0.00	0.1563
	EN	1,514	82.69	9.78	4.62	2.84	0.07	0.4808

Panel (a): N counts evaluated utterances. UTMOSv2 is the mean ± standard deviation; CER applies to ZH and WER to EN, so the other cell is –, and Mixed UTMOSv2 pools both. Panel (b): entries are percentages of the language-specific turns, and mean trunc. tokens uses the same subset.

Table 17. L1 and L2 action prediction.

(a) L1 action prediction

Scope	Decision points	TP	FP	FN	Prec.↑	Rec.↑	F1↑
Overall	156,376	1,397	329	341	80.94	80.38	80.66
Multi	51,636	633	171	197	78.73	76.27	77.48

(b) L2 action prediction

Occupancy	Gold action	Decision points	EM↑	Launch P/R/F1↑	Cancel P/R/F1↑	Invalid ↓
Empty	Keep	34,761	92.5	–	–	1.72
Empty	Launch	7,062	78.3	82/77/79	–	0.30
Single	Keep	40,238	86.7	–	–	5.15
Single	Launch	8,917	70.2	78/72/75	–	0.84
Single	Cancel	9,846	58.4	–	70/61/65	1.02
Single	C+L	3,916	49.3	74/68/71	66/57/61	1.65
Multi (2–3)	Keep	24,713	78.1	–	–	8.11
Multi (2–3)	Launch	5,842	61.1	71/64/67	–	5.56
Multi (2–3)	Cancel	9,276	52.8	–	58/46/51	6.71
Multi (2–3)	C+L	5,453	37.4	66/58/62	52/40/45	7.23
Full (4)	Keep	3,411	74.4	–	–	15.09
Full (4)	Cancel	2,116	48.6	–	54/42/47	6.35
Full (4)	C+L	825	33.9	61/53/57	48/35/41	13.28

(c) L2 invalid-action breakdown

Scope	Other occupied launch	Other idle cancel	Duplicate launch	Duplicate cancel	Total
Overall	1,000	570	410	260	2,240
Multi	860	460	350	215	1,885
Full	150	75	65	35	325

The shared evaluation set contains 1,000 episodes and 156,376 decision points. Precision, recall, F1, EM, and Invalid are percentages; decision points, TP, FP, FN, and panel (c) are counts. C+L denotes cancel+launch. P/R/F1 match the action token and slot; EM executes the sequence in order and permits permutation among initially idle slots. Invalid tokens count as false positives and make EM zero. Panel (b) uses Multi (2–3); Multi in panels (a,c) means at least two in-flight requests and includes Full. In panel (c) each invalid token gets one category, with duplicate errors preceding occupied-launch or idle-cancel errors and Full occupying all four slots.

Table 18. Complete FDB-v1 turn-taking and FDB-v1.5 overlap-handling results.

Task	Metric	Reference systems					AdaptDuplex training						
								Curriculum					
		Moshi†	Freeze-Omni†	Qwen3-Omni +VAD	MiniCPM-o 4.5	Duplex Omni	Mixed SFT	Fixed 0.48 s	Fixed 0.64 s	Fixed 0.96 s	Dynamic	+ Joint SFT	+ RL (GRPO)
(a) FDB-v1: turn-taking													
Pause	Syn. TOR↓	0.985	0.642	0.993	0.270	0.116		0.131	0.130	0.125	0.128	0.126	0.123
	Can. TOR↓	0.980	0.481	0.949	0.463	0.097		0.431	0.406	0.378	0.299	0.291	0.284
Back-channel	TOR↓	1.000	0.636	0.344	0.273	0.491		0.346	0.335	0.327	0.334	0.331	0.327
	Freq.↑	0.001	0.001	0.018	0.020	0.034		0.041	0.040	0.039	0.040	0.040	0.041
	JSD↓	0.957	0.997	0.879	0.902	0.847		0.824	0.823	0.818	0.821	0.818	0.815
Smooth turn	TOR↑	0.941	0.336	0.992	0.882	0.193		0.992	0.980	0.958	0.982	0.984	0.985
	Resp. lat.↓	0.265	0.953	3.358	1.222	0.000‡		0.052	0.079	0.112	0.085	0.079	0.076
Inter-ruption	TOR↑	1.000	0.867	0.995	0.985	0.746		0.985	0.983	0.985	0.985	0.986	0.986
	GPT-4o↑	0.765	3.615	4.824	4.497	4.222		4.168	4.174	4.216	4.512	4.543	4.570
	Resp. lat.↓	0.257	1.409	3.380	1.408	0.158		0.104	0.163	0.234	0.131	0.119	0.113
(b) FDB-v1.5: overlap handling													
Interruption	Respond↑	0.50	0.72	0.97	0.82	0.67	0.62	0.82	0.78	0.76	0.79	0.83	0.85
	Resume↓	0.26	0.12	0.00	0.10	0.12	0.13	0.10	0.11	0.12	0.11	0.09	0.08
	Uncertain↓	0.00	0.03	0.03	0.01	0.00	0.01	0.02	0.00	0.00	0.01	0.00	0.00
	Unknown↓	0.25	0.13	0.01	0.07	0.22	0.24	0.08	0.11	0.12	0.09	0.08	0.07
	Stop lat.↓	1.16	1.42	0.09	2.49	2.01	2.86	1.19	1.81	2.61	1.32	1.20	1.08
	Resp. lat.↓	1.47	1.35	3.70	1.86	0.36	1.25	0.33	0.42	0.58	0.34	0.31	0.27
Backchannel	Respond↓	0.02	0.07	0.34	0.01	0.01	0.00	0.01	0.01	0.01	0.01	0.00	0.00
	Resume↑	0.06	0.80	0.00	0.46	0.28	0.41	0.52	0.54	0.56	0.55	0.55	0.57
	Uncertain↓	0.00	0.02	0.27	0.00	0.02	0.01	0.01	0.00	0.00	0.00	0.00	0.01
	Unknown↓	0.92	0.11	0.40	0.53	0.69	0.58	0.46	0.45	0.43	0.44	0.45	0.42
	Stop lat.↑	0.42	0.66	–	0.76	0.80	0.86	0.82	0.84	0.85	0.84	0.86	0.89
	Resp. lat.↓	3.00	2.16	2.73	2.34	0.82	1.77	0.32	0.66	1.14	0.43	0.32	0.31
Talking to others	Respond↓	0.20	0.58	0.96	0.64	0.15	0.47	0.53	0.51	0.50	0.50	0.47	0.46
	Resume↑	0.19	0.25	0.00	0.20	0.36	0.26	0.29	0.30	0.33	0.33	0.35	0.41
	Uncertain↑	0.02	0.00	0.01	0.02	0.01	0.03	0.04	0.05	0.01	0.03	0.04	0.05
	Unknown↓	0.59	0.15	0.03	0.14	0.47	0.24	0.14	0.14	0.16	0.14	0.14	0.08
	Stop lat.↑	0.87	1.39	0.10	1.71	1.87	1.89	1.88	1.90	1.95	1.89	1.91	1.87
	Resp. lat.↓	2.38	1.32	3.16	1.85	1.95	1.53	0.32	0.32	0.34	0.33	0.32	0.29
Background speech	Respond↓	0.21	0.63	0.98	0.43	0.05	0.47	0.50	0.44	0.37	0.42	0.40	0.36
	Resume↑	0.07	0.25	0.00	0.36	0.07	0.23	0.33	0.42	0.46	0.42	0.45	0.42
	Uncertain↑	0.01	0.01	0.00	0.02	0.00	0.01	0.02	0.01	0.00	0.01	0.02	0.06
	Unknown↓	0.71	0.11	0.02	0.19	0.89	0.29	0.15	0.13	0.17	0.15	0.13	0.16
	Stop lat.↑	0.54	0.98	0.09	1.46	1.50	1.46	1.44	1.47	1.59	1.51	1.56	1.60
	Resp. lat.↓	1.62	1.60	3.19	1.61	0.74	2.07	0.30	0.39	0.59	0.36	0.33	0.31

(a): Freq. in events/s; GPT-4o scores 0–5; latencies in seconds. GPT-4o score and all latencies conditional on TO=1. Mixed SFT is not evaluated on FDB-v1.
‡Negative smooth-turn latencies clipped to zero; excluded from best-value highlighting. †From FDB-v1 [14] Table III; latencies not hardware-matched.
(b): Respond/Resume/Uncertain/Unknown are rates (0–1); Stop/Response are latencies (s). –: no qualifying event. †From FDB-v1.5 [15] Table 2; latencies are not hardware-matched.

Table 19. Execution statistics for serialization formats and window policies.

Format	Window policy	Group	Workload		Comp. Cost		Exec. Time (ms)				
			Windows	Input (min)	GPU time (s)	GPU-s/min ↓	Mean	P50↓	P90↓	P95↓	P99↓
JSON-style	Fixed 0.48 s	All	3,724	29.792	2,833.331	95.104	760.8	789.4	957.6	1,018.7	1,164.2
		SPEAK	2,503	20.024	2,177.410	108.740	870.0	844.8	1,041.2	1,116.5	1,269.3
		LISTEN	1,221	9.768	655.921	67.150	537.2	529.6	658.4	711.8	879.4
Canonical	Fixed 0.48 s	All	3,724	29.792	1,439.586	48.321	386.6	418.5	491.2	565.9	672.2
		SPEAK	2,503	20.024	1,127.973	56.331	450.6	433.1	527.9	605.5	689.9
		LISTEN	1,221	9.768	311.613	31.901	255.2	254.3	316.7	359.7	506.7
	Fixed 0.64 s	All	2,790	29.760	1,223.506	41.112	438.5	480.1	581.7	638.5	741.8
		SPEAK	1,901	20.277	985.681	48.610	518.5	500.5	614.6	678.5	768.6
		LISTEN	889	9.483	237.825	25.080	267.5	265.5	338.9	384.0	538.3
	Fixed 0.96 s	All	1,856	29.696	993.492	33.455	535.3	595.3	714.6	773.3	868.8
		SPEAK	1,298	20.768	827.248	39.833	637.3	619.1	745.8	802.3	901.2
		LISTEN	558	8.928	166.244	18.621	297.9	293.7	381.1	440.8	613.1
	Dynamic	All	2,430	29.712	1,304.880	43.920	465.2	492.6	603.8	665.1	781.4
		SPEAK	1,660	20.064	1,056.420	52.615	548.7	541.3	658.9	721.6	836.5
		LISTEN	770	9.648	248.460	25.751	285.7	286.4	354.2	399.8	566.9

Input is the duration attributed to each group. Cost = GPU seconds / input minutes; ALL aggregates SPEAK and LISTEN. Exec. time is distinct from interaction latency.

Table 20. L1 and L2 information utilization.

(a) L1 memory utilization

Condition	Fact correct/N	Fact Acc.↑	Pref. correct/N	Pref. Acc.↑	FalseUpd. errors/N	FalseUpd. ↓
Rolling only	41/400	10.25	67/400	16.75	–	–
Gold P + M	251/400	62.75	237/400	59.25	17/200	8.50

(b) L2 external-reasoning utilization

Condition	N	Correct	Behavior Acc.↑	Missing target	Forbidden value
Single	200	173	86.50	27	0
Bind	200	147	73.50	31	27
Compose	200	137	68.50	63	8
Fail	200	163	81.50	21	20
None	200	161	80.50	–	39

(c) Bind counterfactual intervention

Condition	Pairs	Original Acc.↑	Swapped Acc.↑	Pair Acc.↑
Bind	200	73.50	70.50	64.00

All accuracies and FalseUpd. are percentages; other entries are counts. Panel (a) uses 1,000 instances: 400 fact, 400 preference, and 200 no-update cases; the two context conditions are paired on the fact and preference instances. Panel (b) uses 1,000 scenarios, 200 per condition. Behavior Acc. equals one iff all target canaries and no forbidden canaries appear. Missing-target and forbidden-value errors may co-occur. Panel (c) is a paired intervention on the 200 Bind cases: A/B payloads are exchanged while the dialogue, query, and request identities remain fixed, Pair Acc. requires both the original and swapped members to be correct.